Mini Paper — 1.4 Data Types, Data Structures & Boolean Algebra — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Q1. [1+2 = 3] (a) 150 = 128 + 16 + 4 + 2 → **`1001 0110`** *(1)* (b) Split into nibbles: `1001` = 9, `0110` = 6 → **0x96** *(1 for nibble split, 1 for correct answer = 2)* Check: 9×16 + 6 = 144 + 6 = 150 ✓ *Examiner tip: always group binary into nibbles from the RIGHT before converting to hex.*

Q2. [2] 2F → first digit 2 = (2 × 16) = 32 *(1)*; second digit F = 15 → 32 + 15 = **47** *(1) Examiner tip: multiply the left hex digit by 16, then add the right digit's value (A–F = 10–15).*

Q3. [4] (a) 45 = `0010 1101`; 58 = `0011 1010` *(1)* (b) −58: invert `0011 1010` → `1100 0101`, add 1 → **`1100 0110`** *(1)* (c) Add `0010 1101 + 1100 0110`:

```
  0 0 1 0 1 1 0 1   (45)
+ 1 1 0 0 0 1 1 0   (-58)
-----------------
  1 1 1 1 0 0 1 1
```

No carry out of the MSB. *(1)* Result `1111 0011` is negative (MSB = 1); magnitude = invert+1 = `0000 1101` = 13, so answer = **−13** *(1)*. (45 − 58 = −13 ✓) *Examiner tip: subtract by adding the two's complement; remember to discard any carry out of the MSB.*

Q4. [3] (a) Mantissa must start `0.1…`; currently `0.00110`, so shift the point **right by 2** places → `0.110` *(1)*. Moving the point right by 2 reduces the exponent by 2: 3 − 2 = **1**, giving **`0.110 × 2¹`** *(1)*. (b) Normalising removes wasted leading zeros so the mantissa holds the **maximum significant figures / greatest precision** for the available bits *(1)*. *Examiner tip: moving the point right DECREASES the exponent; the overall value must stay the same.*

Q5. [3] (a) Arithmetic shift right by 1 on `1111 0100`: shift right and copy the sign bit (1) in from the left → **`1111 1010`** *(1)*. (b) `1111 1010` = −(invert+1) = −`0000 0110` = **−6** *(1)*. An arithmetic shift **preserves the sign bit**, so a negative two's complement number stays negative and is correctly divided by 2; a logical shift would insert a 0, corrupting the sign and giving a wrong (positive) result *(1)*. *Examiner tip: −12 ÷ 2 = −6 — state both the bit pattern and the denary check.*

Q6. [4] - De Morgan's on the first term: $\neg(A + \bar{B}) = \neg A \cdot \neg(\bar{B})$ = **$\bar{A} \cdot B$** (double negation of $\bar{B}$ → B) *(1 for De Morgan's, 1 for double negation)* - Expression becomes $(\bar{A} \cdot B) + (A \cdot B)$ *(1)* - Factor out B (distributive): $B \cdot (\bar{A} + A) = B \cdot 1$ = **B** *(1)* Final answer: **B** *Examiner tip: with De Morgan's, change the operator AND negate each term, then look for a common factor to absorb.*

Q7. [3]

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

(1 mark per correct pair of rows = up to 2 for the column.) F is 1 only when the inputs **differ**, so it is equivalent to an **XOR** gate *(1)*. *Examiner tip:* $A \cdot \bar{B} + \bar{A} \cdot B$ *is the standard sum-of-products definition of XOR.*

Q8. [4] (a) Tracing (size 5, MOD 5; rear points to next free slot):

Operation	Array (idx 0–4)	front	rear
enqueue P	[P, , , ,]	0	1
enqueue Q	[P, Q, , , _]	0	2
enqueue R	[P, Q, R, ,]	0	3
dequeue	[, *Q, R,* , _]	1	3
enqueue S	[, *Q, R, S,*]	1	4
enqueue T	[_, Q, R, S, T]	1	0

Final array contains **Q, R, S, T** (index 0 free), **front = 1**, **rear = 0** *(1 for correct contents, 1 for front, 1 for rear = 3)*. (b) A circular queue **reuses the space freed by dequeuing** (rear wraps round with MOD), so it does not waste slots at the front like a linear queue *(1)*. *Examiner tip: show the MOD wraparound — rear goes 4 → 0 once it passes the end of the array.*

Q9. [4] (a) BST built by inserting 50, 30, 70, 20, 40, 60:

```
        50
       /  \
     30    70
    /  \   /
  20   40 60
```

(1 for 50 as root with 30 left / 70 right; 1 for 20,40 under 30 and 60 as left child of 70) (b) In-order (left, root, right) visits: **20, 30, 40, 50, 60, 70** *(1)*. This is **ascending sorted order**, demonstrating the BST ordering property (left subtree < node < right subtree) *(1)*. *Examiner tip: an in-order traversal of any BST always outputs the values in ascending order — a quick way to check your tree.*